JUnit Basic Testing Exercises

Exercise 1: Setting Up JUnit

Source Code

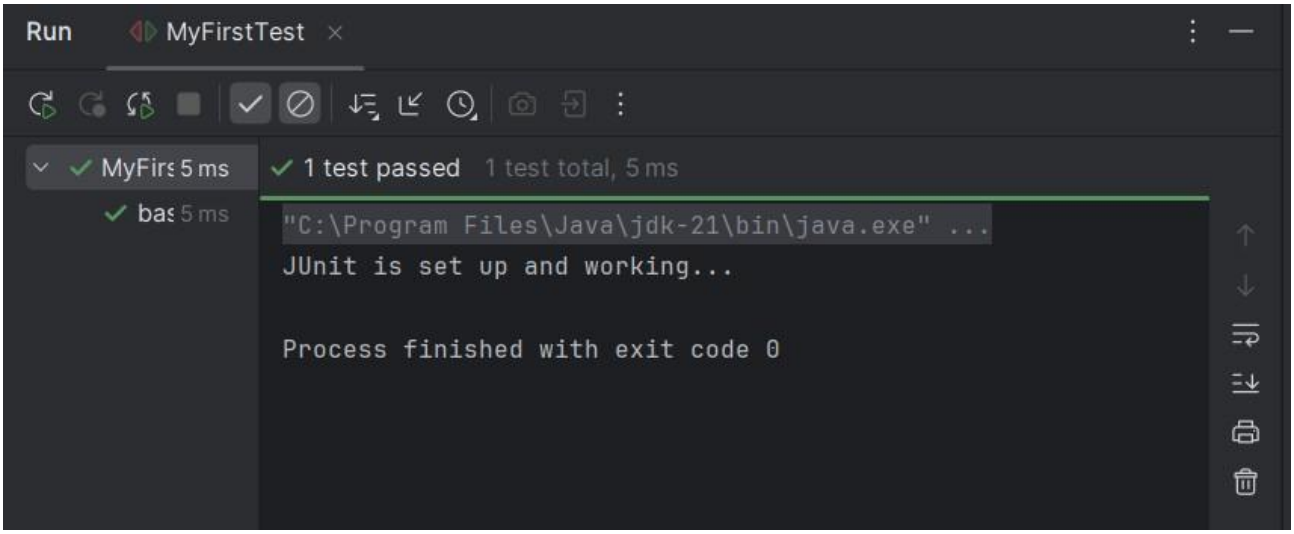
pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>JUnitSetup</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

MyFirstTest.java

```
package com.example;

import org.junit.Test;

public class MyFirstTest {

   @Test    public void basicTest() {
System.out.println("JUnit is set up and working...");
   }
}
```

Output Evidence



Exercise 2: Basic JUnit Test

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>JUnitSetup</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

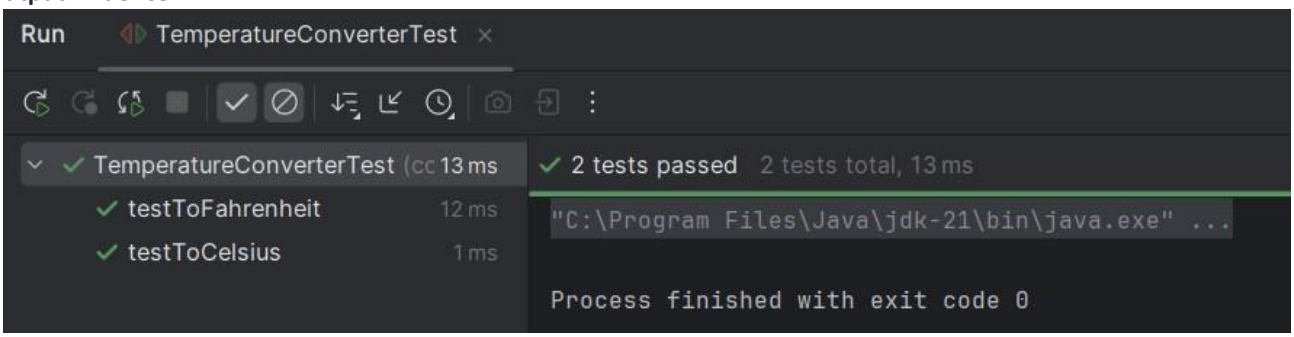
TemperatureConverter.java

```
package com.example;

public class TemperatureConverter {

  public double toFahrenheit(double celsius) {
return (celsius * 9 / 5) + 32;    }

  public double toCelsius(double fahrenheit) {
return (fahrenheit - 32) * 5 / 9;    }
}
```

TemperatureConverterTest.java

```
package com.example;

import org.junit.Test; import static
org.junit.Assert.*;

public class TemperatureConverterTest {    TemperatureConverter

converter = new TemperatureConverter();

  @Test    public void testToFahrenheit() {
double result = converter.toFahrenheit(0);
assertEquals(32.0, result, 0.001);    }

  @Test    public void testToCelsius() {       double
result = converter.toCelsius(212);
assertEquals(100.0, result, 0.001);    }
}
```

Output Evidence



Exercise 3: Assertions In JUnit

Source Code

pom.xml - Libraries and Dependencies

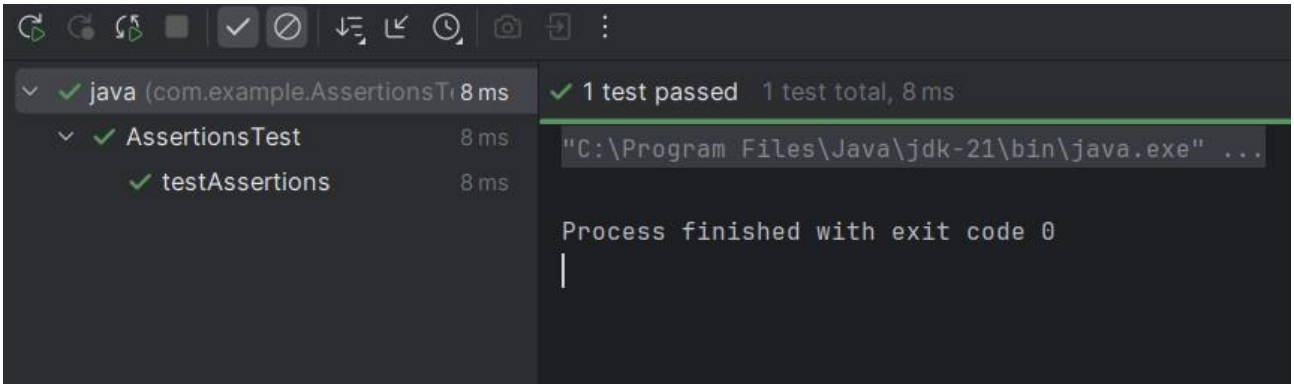
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>JUnitSetup</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

AssertionsTest.java

```
package com.example.AssertionsTest.java;

import org.junit.Test; import static
org.junit.Assert.*;

public class AssertionsTest {

   @Test    public void
testAssertions() {       // Assert
equals       assertEquals(5, 2 + 3);

      // Assert true
assertTrue(5 > 3);

      // Assert false       assertFalse(5
< 3);

      // Assert null       assertNull(null);

      // Assert not null
assertNotNull(new Object());    }
```

}

Output Evidence



Exercise 4: Arrange Act Assert Pattern

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>JUnitSetup</artifactId>
  <version>1.0-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Calculator.java

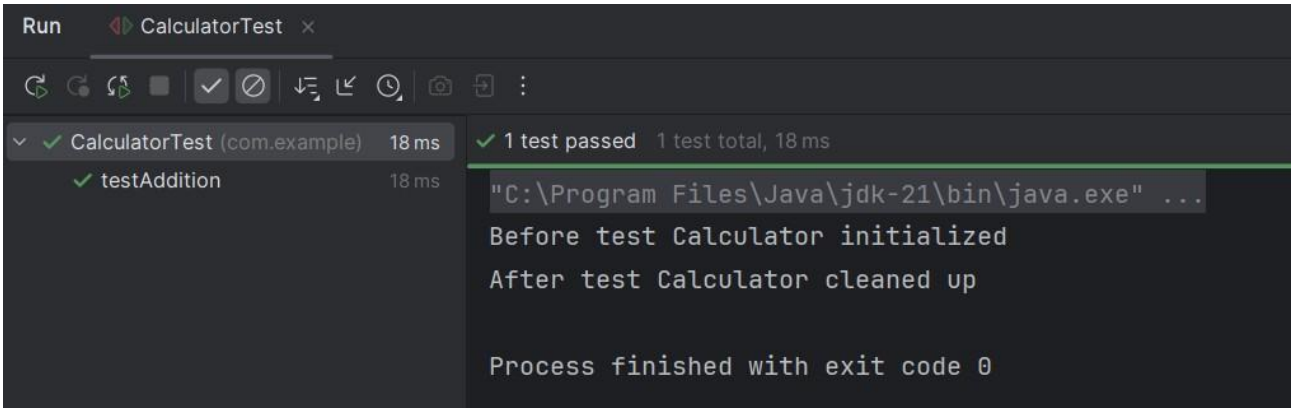
```
package com.example;

public class Calculator {    public int
add(int a, int b) {        return a + b;
}
}
```

CalculatorTest.java

```
package com.example;

import org.junit.Before; import
org.junit.After; import
org.junit.Test; import static
org.junit.Assert.*;

public class CalculatorTest {
```

```
    private Calculator calculator;

    // Arrange setup method     @Before     public void setUp() {
calculator = new Calculator();         System.out.println("Before test
Calculator initialized");
    }

    // Actual test method     @Test
public void testAddition() {

        // Act         int result = calculator.add(2,
3);

        // Assert         assertEquals(5,
result);     }

    // Teardown cleanup method     @After     public void
tearDown() {         calculator = null;         System.out.println("After
test Calculator cleaned up");
    }
}
```

Output Evidence



Mockito Exercises

Exercise 1: Mocking And Stubbing

Source Code

pom.xml - Libraries and Dependencies

```
  <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">     <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoExercise</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
      </dependency>
```

```
        <!-- Mockito -->
        <dependency>
          <groupId>org.mockito</groupId>
          <artifactId>mockito-core</artifactId>
          <version>5.10.0</version>
          <scope>test</scope>
        </dependency>
      </dependencies>
    </project>
```

ExternalApi.java

```
  package com.example;

  public interface ExternalApi {
  String getData();
  }
```

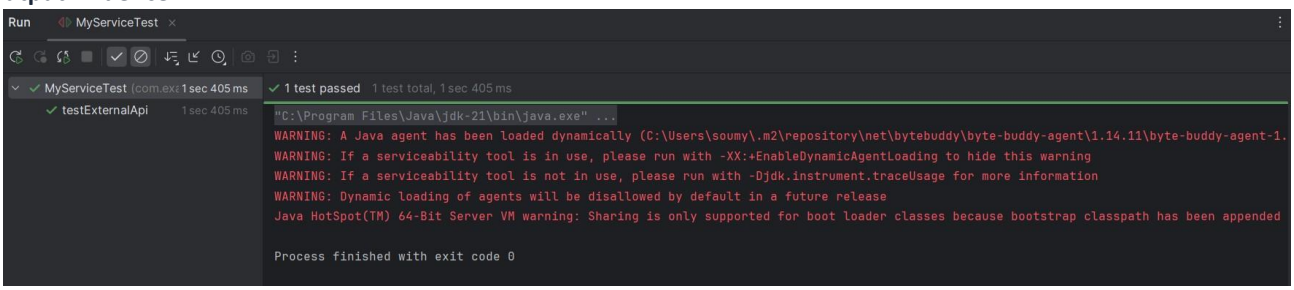
MyService.java

```
  package com.example;

  public class MyService {     private
  final ExternalApi api;

     public MyService(ExternalApi api) {
  this.api = api;     }

     public String fetchData() {
  return api.getData();     }
  }
```

MyServiceTest.java

```
  package com.example;

  import org.junit.Test; import static
  org.mockito.Mockito.*; import static
  org.junit.Assert.*;

  public class MyServiceTest {

     @Test     public void
  testExternalApi() {        // Mock of
  the ExternalApi
        ExternalApi mockApi = mock(ExternalApi.class);

        // Stub the getData() method

  when(mockApi.getData()).thenReturn("Mock Data");

  MyService service = new MyService(mockApi);

        String result = service.fetchData();
  assertEquals("Mock Data", result);     }
  }
```

Output Evidence



Exercise 2: Verifying Interactions

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MockitoVerifyInteraction</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>


    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

ExternalApi.java

```
package com.example;

public interface ExternalApi {
String getData();
}
```

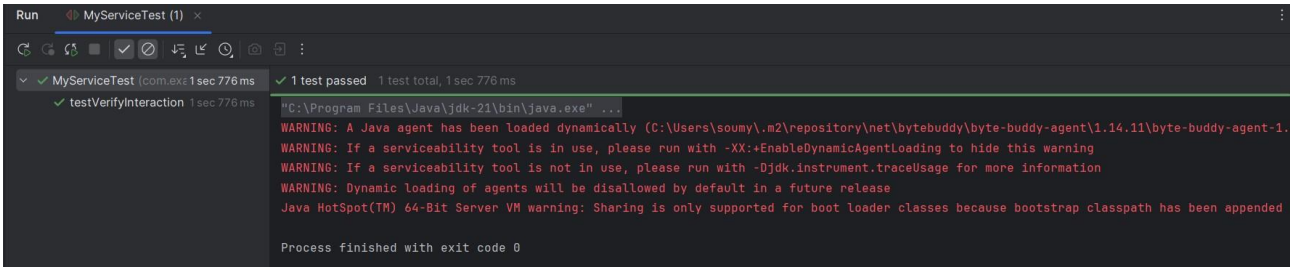
MyService.java

```
package com.example;

public class MyService {

    private final ExternalApi api;

    public MyService(ExternalApi api) {
this.api = api;    }

    public String fetchData() {
return api.getData();    }
}
```

MyServiceTest.java

```
package com.example;

import org.junit.Test; import
org.mockito.Mockito;

import static org.mockito.Mockito.*;

public class MyServiceTest {

    @Test    public void
testVerifyInteraction() {       // Create
mock
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        //Use mock in service       MyService service =
new MyService(mockApi);       service.fetchData();

        // Verify interaction
verify(mockApi).getData();    }
}
```

Output Evidence



Exercise 3: Argument Matching

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
```

```
    <artifactId>MockitoArgumentMatching</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
      </dependency>

      <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
      </dependency>
    </dependencies>
  </project>
```

AlertService.java

```
  package com.example;

  public class AlertService {

    private final Notifier notifier;
    public AlertService(Notifier notifier) {
  this.notifier = notifier;    }

    public void sendAlert(String userId) {        String msg =
  "Your session is about to expire!";
  notifier.send(userId, msg);    }
  }
```

Notifier.java

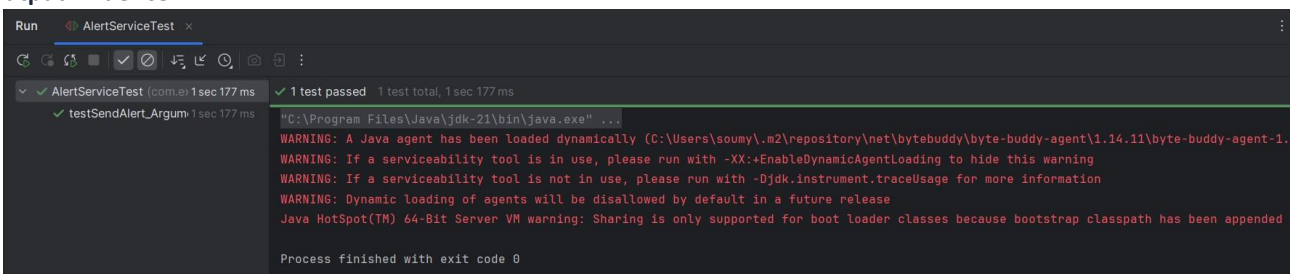
```
  package com.example;

  public interface Notifier {    void send(String
  userId, String message); }
```

AlertServiceTest.java

```
  package com.example;

  import org.junit.Test; import static
  org.mockito.Mockito.*; import static
  org.mockito.ArgumentMatchers.*;

  public class AlertServiceTest {

    @Test
    public void testSendAlert_ArgumentMatching() {

      Notifier mockNotifier = mock(Notifier.class);
      AlertService service = new AlertService(mockNotifier);

      // Act
  service.sendAlert("user_101");
```

```
        //          Verify          using          argument          matchers
    verify(mockNotifier).send(eq("user_101"), contains("expire"));    }
    }
```

Output Evidence



Exercise 4: Handling Void Methods

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MockitoArgumentMatching</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

EmailService.java

```
package com.example;

public interface EmailService {    void sendEmail(String to,
String subject, String body); }
```

UserManager.java

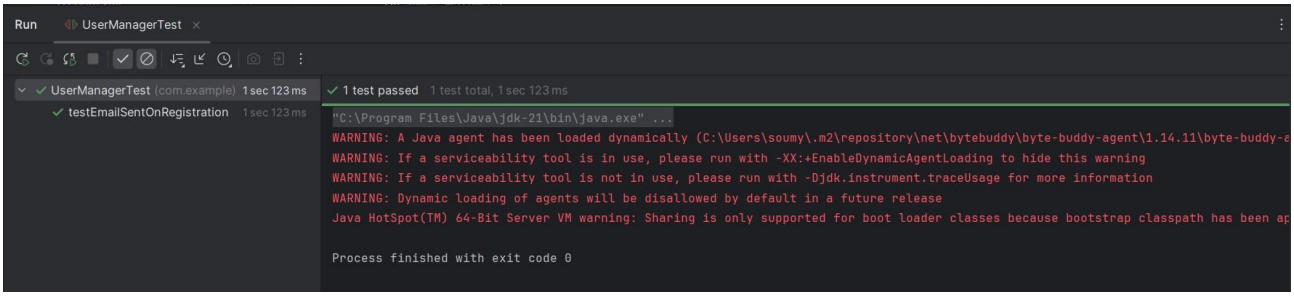
```
package com.example;

public class UserManager {
```

```
    private final EmailService emailService;

    public UserManager(EmailService emailService) {
this.emailService = emailService;    }

    public void registerUser(String email) {        String
subject = "Welcome!";        String body = "Thanks for
registering.";        emailService.sendEmail(email,
subject, body);    }
}
```

UserManagerTest.java

```
package com.example;

import org.junit.Test; import static
org.mockito.Mockito.*;

public class UserManagerTest {

    @Test
    public void testEmailSentOnRegistration() {        EmailService
mockEmailService = mock(EmailService.class);
        UserManager manager = new UserManager(mockEmailService);

        // Call the method under test
manager.registerUser("test@example.com");

        // Verify the void method was called
verify(mockEmailService).sendEmail(
eq("test@example.com"),
eq("Welcome!"),              eq("Thanks for
registering.")        );
    }
}
```

Output Evidence



Exercise 5: Multiple Returns

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.example</groupId>
    <artifactId>MockitoArgumentMatching</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
      </dependency>

      <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
      </dependency>
    </dependencies>
  </project>
```

WeatherApi.java

```
  package com.example;

  public interface WeatherApi {
  String getForecast();
  }
```

WeatherService.java

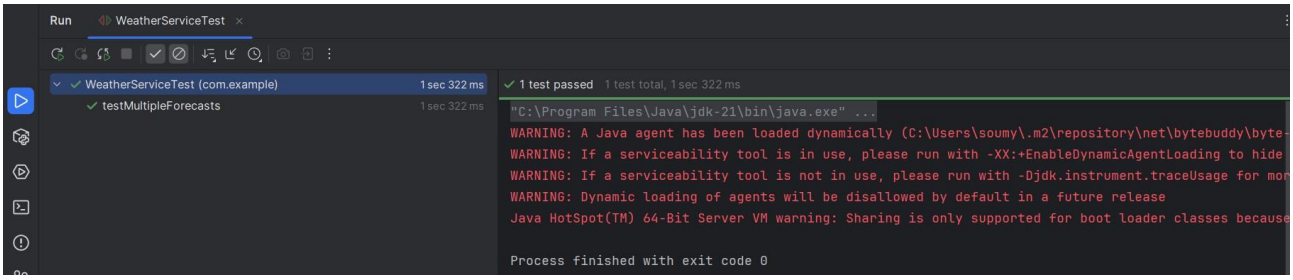
```
  package com.example;

  public class WeatherService {

    private final WeatherApi api;

    public WeatherService(WeatherApi api) {
  this.api = api;    }

    public String[] fetchForecasts() {
  return new String[] {
  api.getForecast(),
  api.getForecast(),
  api.getForecast()       };
    }
  }
```

WeatherServiceTest.java

```
  package com.example;

  import org.junit.Test; import static
  org.junit.Assert.*; import static
  org.mockito.Mockito.*;

  public class WeatherServiceTest {

    @Test
    public void testMultipleForecasts() {

  WeatherApi mockApi = mock(WeatherApi.class);
```

```
        when(mockApi.getForecast())
    .thenReturn("Sunny")
                .thenReturn("Cloudy")
                .thenReturn("Rainy");

        WeatherService service = new WeatherService(mockApi);
        String[] result = service.fetchForecasts();

        //Verify the results        assertArrayEquals(new String[] { "Sunny",
    "Cloudy", "Rainy" }, result);     }
    }
```

Output Evidence



SLF4J Logging Exercises

Exercise 1: Logging Error And Warning

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>LoggingExample</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <!-- SLF4J-->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>1.7.30</version>
    </dependency>

    <!-- Logback -->
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.2.3</version>
    </dependency>
  </dependencies>
</project>
```

LoggingExample.java

```
package com.example;

import org.slf4j.Logger; import
org.slf4j.LoggerFactory;

public class LoggingExample {    private static final Logger logger =
LoggerFactory.getLogger(LoggingExample.class);

  public static void main(String[] args) {
logger.error("This is an ERROR Message");
logger.warn("This is a WARNING Message");    }
}
```

Output Evidence

Output Screenshot

Exercise 2: Parameterized Logging

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">    <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>ParameterizedLoggingExample</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>1.7.30</version>
    </dependency>
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.2.3</version>
    </dependency>
  </dependencies>
</project>
```

ParameterizedLoggingExample.java

```
package com.example;

import org.slf4j.Logger; import
org.slf4j.LoggerFactory;

public class ParameterizedLoggingExample {    private static final
Logger logger =
LoggerFactory.getLogger(ParameterizedLoggingExample.class);

  public static void main(String[] args) {
String user = "Soumyadip Karan";       int age
= 20;
```

```
        logger.info("User {} has logged in", user);
    logger.debug("User {} is {} years old", user, age);     }
    }
```

Output Evidence

Output Screenshot

Exercise 3: Different Appenders

Source Code

pom.xml - Libraries and Dependencies

```
  <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">     <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>AppenderExample</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
      <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>1.7.30</version>
      </dependency>
      <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.2.3</version>
      </dependency>
    </dependencies>
  </project>
```

AppenderExample.java

```
  package com.example;

  import org.slf4j.Logger; import
  org.slf4j.LoggerFactory;

  public class AppenderExample {

    private static final Logger logger = LoggerFactory.getLogger(AppenderExample.class);
```

```
    public static void main(String[] args) {
logger.info("Info message");
logger.error("Error message");     }
} 
```

logback.xml - Logging Configuration

```
<configuration>

    <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <appender name="file" class="ch.qos.logback.core.FileAppender">
        <file>app.log</file>
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="debug">
        <appender-ref ref="console"/>
        <appender-ref ref="file"/>
    </root>

</configuration>
```

Output Evidence